

# Artificial Intelligence Exam Notes

CT 710 (BEI, IV-I) · CT 653 (BCT, III-II) · CT 78506 (BEX, IV-II) · TU, IOE · tiers over 41 papers, 2069–2082 BS

## 7. Applications of AI – Solved Problems

Every problem opens with the question **exactly as the paper prints it**, the paper's own spelling and slips included. · The gate networks (17 papers), the Hebbian AND net (4), the Hopfield weight matrix (1 at 6 marks), the printed forward-propagation network (2 at 8 marks each) and the parse tree (1). · Every weight, net input, activation and matrix entry printed here is recomputed independently by `verify.py`.

### Three things to know before you start.

- **Use bipolar values,  $\pm 1$ , not 0 and 1.** With binary inputs a 0 contributes nothing to any weight update, so the Hebb net for AND *cannot be learnt at all* and the perceptron takes six epochs instead of two. Every worked answer below uses  $\pm 1$  and says so in its first line.
- **“Design” and “train” are different questions.** Design a net for AND: choose weights, verify the four rows, done in five lines. Construct it using the perceptron / Hebb algorithm and show each calculation: run the table. Read which one the paper asked – 77 Ch says “show each calculation performed” and will not accept a designed answer.
- **Always finish with the verification table.** Four rows, *net* and *y* against the target. It is two minutes, it catches every arithmetic slip, and the marks for it are free.

### 7.N.1 Problem 1.1 – McCulloch-Pitts neuron for AND and OR

What is a McCulloch/Pitts neural network? Explain it with reference to the AND gate 8 10

PIN 2/41 75 Ba 8, 76 Ba 10 · both papers then ask *why it cannot do XOR*, which is Problem 1.4 · the MP neuron's weights are chosen, not learnt, so this problem is solved by inequalities, never by a training table

#### QUESTION AS PRINTED 75 Ba · Q9 · 8

What is a McCulloch/Pitts neural network? Explain it with reference to AND gate. Justify that it cannot be applied to Exclusive OR gate.

#### Step 1 Write the network and the firing condition

- Two inputs  $x_1, x_2$ , weights  $w_1, w_2$ , threshold  $T$ . The neuron fires ( $y = 1$ ) exactly when

$$x_1 w_1 + x_2 w_2 > T$$

- **Nothing is learnt.** The task is to find one  $(w_1, w_2, T)$  that makes the four rows come out right.

#### Step 2 One inequality per row of the truth table

| $x_1$ | $x_2$ | AND target | Condition       |       |
|-------|-------|------------|-----------------|-------|
| 0     | 0     | 0          | $0 \leq T$      | (i)   |
| 0     | 1     | 0          | $w_2 \leq T$    | (ii)  |
| 1     | 0     | 0          | $w_1 \leq T$    | (iii) |
| 1     | 1     | 1          | $w_1 + w_2 > T$ | (iv)  |

#### Step 3 Solve them

- (ii) and (iii) say each weight alone must *not* reach the threshold; (iv) says the two together must pass it. So pick  $w_1 = w_2 = w$  and **any  $T$  with  $w \leq T < 2w$** .
- Take  $w_1 = w_2 = 1$  and  $T = 1.5$ . Or, as the deck does,  $w_1 = w_2 = 0.7$  and  $T = 1$ .

#### Step 4 Verify all four rows

| $x_1$ | $x_2$ | $net = x_1 + x_2$ | $net > 1.5?$ | $y$ |
|-------|-------|-------------------|--------------|-----|
| 0     | 0     | 0                 | no           | 0 ✓ |
| 0     | 1     | 1                 | no           | 0 ✓ |
| 1     | 0     | 1                 | no           | 0 ✓ |
| 1     | 1     | 2                 | yes          | 1 ✓ |

**AND.**  $w_1 = w_2 = 1, T = 1.5$ .

#### The OR gate, same method

- Rows:  $0 \leq T; w_2 > T; w_1 > T; w_1 + w_2 > T$ . Now *either* weight alone must pass, so  $0 \leq T < w$ .
- Take  $w_1 = w_2 = 1, T = 0.5$ :  $net = 0$  (no), 1 (yes), 1 (yes), 2 (yes). ✓

#### NOT, for completeness

- One inhibitory input:  $w = -1, T = -0.5$ .  
 $x = 0 \rightarrow net = 0 > -0.5 \rightarrow y = 1$ ;  
 $x = 1 \rightarrow net = -1 \rightarrow y = 0$ . ✓
- **AND, OR and NOT together make every Boolean function**, so a *network* of MP neurons is universal even though one neuron is not. That sentence is what 76 Ba's “propose a neural network model” wants.

### 7.N.2 Problem 1.2 — train a perceptron to be an AND gate

Write down the perceptron algorithm and use it to construct a network which performs like a logical AND operation, showing each calculation **2+6 2+2+4**

**HOT 4/41** 77 Ch **2+6**, 73 Bh **2+6**, 78 Po **2+2+4**, 79 Jth **1+6** · 77 Ch says “show each calculation performed”, so the table below is the answer and a designed set of weights is not · 79 Jth says “acts as logic gates”, so give AND and add OR from Problem 1.3

**QUESTION AS PRINTED** 77 Ch · Q9 · **2+6**

Why do you need Multilayer Perceptron Neural Networks? Write down the perceptron algorithm and use it to Construct Network which perform like Logical AND Operation and show each calculation performed.

#### Step 1 Set up

- **Bipolar** inputs and targets,  $\pm 1$ . Bias input fixed at +1.
- Learning rate  $\alpha = 1$ , threshold  $\theta = 0$ .
- Start with  $w_1 = w_2 = b = 0$ .
- **Activation:**  $y = 1$  if  $net > 0$ ,  $y = 0$  if  $net = 0$ ,  $y = -1$  if  $net < 0$ .
- **Update, only when  $y \neq t$ :**

$$w_i \leftarrow w_i + \alpha t x_i, \quad b \leftarrow b + \alpha t$$

- **$net = 0$  counts as an error**, because  $y = 0$  never equals a target of  $\pm 1$ . That is what makes the very first row update.

Truth table, in bipolar form

| $x_1$ | $x_2$ | AND | $t$ |
|-------|-------|-----|-----|
| 1     | 1     | 1   | +1  |
| 1     | -1    | 0   | -1  |
| -1    | 1     | 0   | -1  |
| -1    | -1    | 0   | -1  |

#### Step 2 Epoch 1

| $(x_1, x_2)$ | $net$ | $y$ | $t$ | Update                    | $w_1$ | $w_2$ | $b$ |
|--------------|-------|-----|-----|---------------------------|-------|-------|-----|
| start        |       |     |     |                           | 0     | 0     | 0   |
| (1, 1)       | 0     | 0   | +1  | $y \neq t$ , add $t x_i$  | 1     | 1     | 1   |
| (1, -1)      | 1     | 1   | -1  | $y \neq t$                | 0     | 2     | 0   |
| (-1, 1)      | 2     | 1   | -1  | $y \neq t$                | 1     | 1     | -1  |
| (-1, -1)     | -3    | -1  | -1  | <b>correct, no change</b> | 1     | 1     | -1  |

Sample working, row 2:  $net = b + w_1 x_1 + w_2 x_2 = 1 + 1(1) + 1(-1) = 1$ , so  $y = 1$  but  $t = -1$ . Then  $w_1 = 1 + (-1)(1) = 0$ ,  $w_2 = 1 + (-1)(-1) = 2$ ,  $b = 1 + (-1) = 0$ .

#### Step 3 Epoch 2 — the verification pass

| $(x_1, x_2)$ | $net = -1 + x_1 + x_2$ | $y$ | $t$ |   |
|--------------|------------------------|-----|-----|---|
| (1, 1)       | $-1 + 1 + 1 = 1$       | 1   | +1  | ✓ |
| (1, -1)      | $-1 + 1 - 1 = -1$      | -1  | -1  | ✓ |
| (-1, 1)      | $-1 - 1 + 1 = -1$      | -1  | -1  | ✓ |
| (-1, -1)     | $-1 - 1 - 1 = -3$      | -1  | -1  | ✓ |

**Answer.** No weight changed in epoch 2, so the perceptron has converged after one epoch of updates:  $w_1 = 1$ ,  $w_2 = 1$ ,  $b = -1$ . Decision line  $x_1 + x_2 = 1$ .

### 7.N.3 Problem 1.3 — train a perceptron to be an OR gate

How can we design a neural network that acts as an OR gate? **2+7**

72 Ma **2+7** · **OR converges even faster than AND** — one row of updates and it is done · only the target column changes from Problem 1.2, so if you have written that one, say so and change the column

**QUESTION AS PRINTED** 72 Ma · Q8 · **2+7**

What do you understand by a perceptron? How can we design a neural network that acts as an OR gate.

## Same setup, one column different

| $(x_1, x_2)$ | $net$ | $y$ | $t$ | $w_1$ | $w_2$ | $b$ |
|--------------|-------|-----|-----|-------|-------|-----|
| start        |       |     |     | 0     | 0     | 0   |
| (1, 1)       | 0     | 0   | +1  | 1     | 1     | 1   |
| (1, -1)      | 1     | 1   | +1  | 1     | 1     | 1   |
| (-1, 1)      | 1     | 1   | +1  | 1     | 1     | 1   |
| (-1, -1)     | -1    | -1  | -1  | 1     | 1     | 1   |

- **Only the first row ever updates.** After it, every remaining pattern is already classified correctly, so the weights are untouched.
- Epoch 2 confirms:  $net = 3, 1, 1, -1$  against targets  $+1, +1, +1, -1$ . ✓

**Answer.**  $w_1 = 1, w_2 = 1, b = +1$ . Decision line  $x_1 + x_2 = -1$ .

## AND and OR side by side, which is worth a sentence

|                   | AND                | OR                 |
|-------------------|--------------------|--------------------|
| $w_1, w_2$        | 1, 1               | 1, 1               |
| Bias $b$          | -1                 | +1                 |
| Fires when        | both inputs are +1 | at least one is +1 |
| Epochs of updates | 1 (three updates)  | 1 (one update)     |

- **The weights are identical; only the bias moves.** The bias slides the same decision line across the square: with  $b = -1$  it cuts off the single corner (1, 1), with  $b = +1$  it cuts off the single corner (-1, -1).
- That is the cleanest statement of what a bias is for, and it is the answer to 79 Jth's "can we design a neural network that acts as logic gates": yes, and the same network does both, with one number changed.
- **NAND** is AND with the targets flipped, giving  $w_1 = w_2 = -1, b = +1$ ; and **NOR** is OR flipped. Both are needed in Problem 1.4.

## 7.N.4 Problem 1.4 – XOR: the proof, and the network that does work

Is a single perceptron sufficient to realize a network behaving as an XOR gate? Justify. How can we design one that does?

6 10 1+8 2+6 1+7

**TOP 8/41** 73 Ma 1+7, 74 Ma 10, 74 Bh 1+8, 82 Bh 2+6, 79 Ash 1+8, 77 Ch 2+6, 76 Ba 10, 75 Ba 8 · eight papers, and the answer has two halves: prove one neuron cannot, then build one that can. Doing only the first half costs half the marks, and doing only the second costs more

**QUESTION AS PRINTED** 79 Ash · Q8 · 1+8

What is perceptron? Is single perceptron sufficient to realize a network behaving as XOR gate? Justify.

**Part 1 Why one perceptron cannot: the geometric answer**

- Plot the four inputs on the unit square. XOR outputs 1 at (0, 1) and (1, 0), and 0 at (0, 0) and (1, 1).
- The two 1s are at **diagonally opposite** corners, and so are the two 0s. **No straight line puts one pair on one side and the other pair on the other.**
- A single neuron computes  $w_1x_1 + w_2x_2 + b$  and thresholds it, and the boundary of a threshold is exactly a straight line. So it cannot be done. **Draw the square with the two attempted lines.**

**Part 1 again The algebraic proof, if the paper says “justify”**

- Assume  $w_1, w_2, b$  exist with output 1 iff  $w_1x_1 + w_2x_2 + b > 0$ :
  - (i)  $(0, 0) \rightarrow 0: b \leq 0$
  - (ii)  $(0, 1) \rightarrow 1: w_2 + b > 0$
  - (iii)  $(1, 0) \rightarrow 1: w_1 + b > 0$
  - (iv)  $(1, 1) \rightarrow 0: w_1 + w_2 + b \leq 0$
- Adding (ii) and (iii):  $w_1 + w_2 + 2b > 0$ , so  $w_1 + w_2 > -2b$ .
- But (iv) gives  $w_1 + w_2 \leq -b$ . Hence  $-2b < -b$ , i.e.  $b > 0$ , contradicting (i).
- No such weights exist. ■**
- Say what this does not mean:** it is a limit on *one* neuron, not on neural networks. XOR is not a hard function; it is a *non-linearly-separable* one.

**Part 2 The two-layer network that does work**

- The idea in one line:**  
 $x_1 \oplus x_2 = (x_1 \vee x_2) \wedge \neg(x_1 \wedge x_2)$ . So build OR and AND in a hidden layer and combine them at the output.
- The network** (binary inputs, step activation, fire if  $net > 0$ ):
  - Hidden  $h_1 = \text{OR}$ :  $w = (1, 1), b = -0.5$
  - Hidden  $h_2 = \text{AND}$ :  $w = (1, 1), b = -1.5$
  - Output  $y$ :  $w = (+1 \text{ from } h_1, -1 \text{ from } h_2), b = -0.5$

**Verification, all four rows**

| $x_1$ | $x_2$ | $net_1$ | $h_1$ | $net_2$ | $h_2$ | $y$      |
|-------|-------|---------|-------|---------|-------|----------|
| 0     | 0     | -0.5    | 0     | -1.5    | 0     | -0.5 → 0 |
| 0     | 1     | 0.5     | 1     | -0.5    | 0     | 0.5 → 1  |
| 1     | 0     | 0.5     | 1     | -0.5    | 0     | 0.5 → 1  |
| 1     | 1     | 1.5     | 1     | 0.5     | 1     | -0.5 → 0 |

**Answer.** A single perceptron cannot. A **2-2-1 multilayer perceptron** can: hidden units computing OR and AND, output computing  $h_1 - h_2 - 0.5$ . Output column matches XOR exactly.

- Why it works, geometrically:** each hidden unit draws one straight line, and the output unit takes the strip *between* them. Two lines can isolate a diagonal pair; one cannot.
- And why it needs back-propagation to be learnt:** the hidden units have no targets of their own, so no perceptron rule can train them. This is the answer to 77 Ch's “why do you need multilayer perceptron networks” and to 79 Ch's “when do we need the back propagation algorithm”. One step of it is Problem 2.2.

**7.N.5 Problem 1.5 – a Hebbian network for AND**

**Define Hebbian learning. Use the Hebbian learning algorithm to construct a network which performs the AND function**

2+6 1+7 3+7 6+3

**HOT 4/41** 73 Bh 3+7, 81 Ba 2+6, 78 Ba 1+7, 78 Ch 6+3 · **one pass, no iteration, no error term** — that is what makes it different from Problem 1.2 and it is what the examiner checks · 78 Ch also asks *is Hebbian learning supervised? Justify*

**QUESTION AS PRINTED** 81 Ba · Q8 · 2+6

What is perception? Construct a Hebbian Network that performs like AND Gate.

**Step 1 The rule**

- Bipolar inputs and targets; bias input fixed at +1; all weights start at 0.
- For each training pair, **set the output to the target** and update:

$$w_i \leftarrow w_i + x_i t, \quad b \leftarrow b + t$$

- **No learning rate, no comparison, no second pass.** Every pattern updates, whether the current weights would have got it right or not.

**Step 2 One pass through the four patterns**

| $(x_1, x_2)$ | $t$ | Working                        | $w_1$ | $w_2$ | $b$ |
|--------------|-----|--------------------------------|-------|-------|-----|
| start        |     |                                | 0     | 0     | 0   |
| (1, 1)       | +1  | $0+(1)(1), 0+(1)(1), 0+1$      | 1     | 1     | 1   |
| (1, -1)      | -1  | $1+(1)(-1), 1+(-1)(-1), 1-1$   | 0     | 2     | 0   |
| (-1, 1)      | -1  | $0+(-1)(-1), 2+(-1)(-1), 0-1$  | 1     | 1     | -1  |
| (-1, -1)     | -1  | $1+(-1)(-1), 1+(-1)(-1), -1-1$ | 2     | 2     | -2  |

**Step 3 Verify**

| $(x_1, x_2)$ | $net = 2x_1 + 2x_2 - 2$ | $y$ | $t$ |   |
|--------------|-------------------------|-----|-----|---|
| (1, 1)       | $2 + 2 - 2 = 2$         | 1   | +1  | ✓ |
| (1, -1)      | $2 - 2 - 2 = -2$        | -1  | -1  | ✓ |
| (-1, 1)      | $-2 + 2 - 2 = -2$       | -1  | -1  | ✓ |
| (-1, -1)     | $-2 - 2 - 2 = -6$       | -1  | -1  | ✓ |

Answer.  $w_1 = 2, w_2 = 2, b = -2$ , from one pass. Decision line  $x_1 + x_2 = 1$  – the same line the perceptron found in Problem 1.2, at twice the scale.

**Is Hebbian learning supervised? (78 Ch 3)**

- **Yes, as used here.** The update contains  $t$ , the target, so the network is being told the right answer for every pattern. Without  $t$  there is no AND net.
- **But Hebb's original postulate is not.** *Neurons that fire together wire together* describes a purely local, unsupervised change in a synapse; nothing in it mentions a desired output.
- So the honest answer is both sentences: the principle is unsupervised, the *Hebb net for pattern association* is supervised because it substitutes the target for the output.
- **Why bipolar is compulsory here.** With binary values, the pattern (0, 0) contributes  $0 \times t = 0$  to both weights, so it can never push the weights away from a wrong answer. Run the same table in binary and the AND net comes out wrong. Say this if you have room; it is the trap the question is built on.

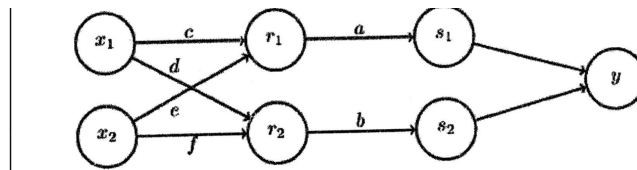
**7.N.6 Problem 2.1 – forward propagation through the printed network**

Perform forward propagation, and update the weights of the second hidden layer, a and b 8

PIN 2/41 82 Ka 8, 81 Ch 8 · the same question printed twice, with the same figure · the paper gives no target value and no learning rate, so both are assumed below and stated · the interesting part is what happens to c, d, e and f

QUESTION AS PRINTED 82 Ka · Q7 · 8 · identical at 81 Ch · Q9

Below is a neural network with weights a, b, c, d, e, and f. The input are  $x_1$  and  $x_2$ . The first hidden layer computes  $r_1 = \max(c \cdot x_1 + d \cdot x_2, 0)$  and  $r_2 = \max(e \cdot x_1 + f \cdot x_2, 0)$ . The second hidden layer and third layer computes  $s_1, s_2$  and  $y$  using sigmoid activation function. Suppose the network has inputs  $x_1 = -1$  and  $x_2 = -1$ . The weight values are  $a = 1, b = 1, c = 4, d = 1, e = 2$  and  $f = 2$ . Perform forward propagation. And update weight for second hidden layer for a and b.



**Step 1 Read the network off the figure**

- $x_1 \rightarrow r_1$  with weight  $c$ ,  $x_2 \rightarrow r_1$  with weight  $e$ ;  $x_1 \rightarrow r_2$  with  $d$ ,  $x_2 \rightarrow r_2$  with  $f$ .
- $r_1 \rightarrow s_1$  and  $r_2 \rightarrow s_2$  apply the **sigmoid**.
- $s_1 \rightarrow y$  with weight  $a$ ,  $s_2 \rightarrow y$  with  $b$ , then sigmoid again.  
**No biases anywhere**, which the figure confirms.
- $\max(z, 0)$  is the **ReLU**;  $\sigma(z) = 1/(1 + e^{-z})$ .

**Step 2 First hidden layer**

$$r_1 = \max(cx_1 + ex_2, 0) = \max(4(-1) + 2(-1), 0) \\ = \max(-6, 0) = 0$$

$$r_2 = \max(dx_1 + fx_2, 0) = \max(1(-1) + 2(-1), 0) \\ = \max(-3, 0) = 0$$

**Step 3 Second hidden layer and output**

$$s_1 = \sigma(r_1) = \sigma(0) = \frac{1}{1+e^0} = 0.5$$

$$s_2 = \sigma(r_2) = \sigma(0) = 0.5$$

$$net_y = as_1 + bs_2 = 1(0.5) + 1(0.5) = 1$$

$$y = \sigma(1) = \frac{1}{1+e^{-1}} = 0.7311$$

**Forward pass.**  $r_1 = r_2 = 0$ ,  $s_1 = s_2 = 0.5$ ,  $y = 0.7311$ .

**Step 4 Update a and b**

The paper gives no target and no learning rate. Assume  $t = 1$  and  $\eta = 0.1$ , and say so — the method is what is marked, and any other pair of values substitutes into the same three lines.

- Loss  $E = \frac{1}{2}(y - t)^2$ . Output delta:

$$\delta = (y - t)y(1 - y)$$

$$= (0.7311 - 1)(0.7311)(0.2689) = -0.05288$$

- Gradients, since the input to both weights is 0.5:

$$\frac{\partial E}{\partial a} = \delta s_1 = -0.05288 \times 0.5 = -0.02644$$

and  $\partial E/\partial b$  is the same, because  $s_1 = s_2$ .

- Update:

$$a \leftarrow a - \eta \frac{\partial E}{\partial a} = 1 - 0.1(-0.02644) = 1.00264$$

$$b \leftarrow b - \eta \frac{\partial E}{\partial b} = 1.00264$$

- **Both weights rise**, and by the same amount, because the output was below the target and both units fed it the same 0.5.

**The part worth extra marks: c, d, e and f cannot be updated at all**

- The gradient reaching  $c$  runs through  $r_1 = \max(-6, 0)$ . The derivative of ReLU at a negative input is 0, so  $\partial E/\partial c = \partial E/\partial d = \partial E/\partial e = \partial E/\partial f = 0$ .
- Both hidden units are **dead for this input**: no gradient flows past them, so the first layer learns nothing from this example. This is the textbook **dying-ReLU** problem, and it is precisely why the question asks you to update only  $a$  and  $b$ .
- Saying that explicitly is what turns a correct arithmetic answer into a complete one.

**7.N.7 Problem 2.2 — one full step of back-propagation**

**Explain the back-propagation algorithm with an example** 4 2+6 3+5 4+4

**TOP 8/41** 70 Bh 4+4, 72 Ash 2+4+4, 70 Ma 3+5, 80 Bh 2+2+4, 74 Bh 4, 79 Ch 2+6, 71 Ma 3+5, 72 Ash 3+5 · **no paper supplies numbers**, so this is the standard XOR network with the weights every deck uses · one input pattern and one step is enough for 6 marks; do not attempt a full epoch

**PROBLEM** not a PYQ · the 2-2-1 XOR network, with the weights the lecture decks use

Train the network below on the XOR pattern  $x = (1, 1)$ , target  $t = 0$ , for one step. Sigmoid activation, loss  $E = \frac{1}{2}(y - t)^2$ , learning rate  $\eta = 0.1$ . Initial weights  $w_{13} = 0.5$ ,  $w_{14} = 0.9$ ,  $w_{23} = 0.4$ ,  $w_{24} = 1.0$ ,  $w_{35} = -1.2$ ,  $w_{45} = 1.1$ ; biases  $b_3 = -0.8$ ,  $b_4 = 0.1$ ,  $b_5 = -0.3$ .

**Step 1 Forward pass**

$$net_3 = x_1 w_{13} + x_2 w_{23} + b_3 = 0.5 + 0.4 - 0.8 = 0.1$$

$$h_3 = \sigma(0.1) = \frac{1}{1+e^{-0.1}} = \mathbf{0.5250}$$

$$net_4 = x_1 w_{14} + x_2 w_{24} + b_4 = 0.9 + 1.0 + 0.1 = 2.0$$

$$h_4 = \sigma(2.0) = \mathbf{0.8808}$$

$$net_5 = h_3 w_{35} + h_4 w_{45} + b_5$$

$$= 0.5250(-1.2) + 0.8808(1.1) - 0.3 = 0.0389$$

$$y_5 = \sigma(0.0389) = \mathbf{0.5097}$$

$$- \text{Error } E = \frac{1}{2}(0.5097 - 0)^2 = \mathbf{0.1299}.$$

**Step 2 Output delta**

$$\delta_5 = (y_5 - t) y_5 (1 - y_5)$$

$$= (0.5097)(0.5097)(0.4903) = \mathbf{0.12738}$$

$y(1 - y)$  is the derivative of the sigmoid, and the forward pass already gave you  $y$  – that is the whole reason sigmoid is used.

**Step 3 Hidden deltas**

$$\delta_3 = \delta_5 w_{35} h_3 (1 - h_3)$$

$$= 0.12738(-1.2)(0.5250)(0.4750) = \mathbf{-0.03812}$$

$$\delta_4 = \delta_5 w_{45} h_4 (1 - h_4)$$

$$= 0.12738(1.1)(0.8808)(0.1192) = \mathbf{0.01471}$$

- $\delta_3$  is negative because  $w_{35}$  is: unit 3 pushes the output down, so to reduce an output that is too high you must push unit 3 up. The sign carries real information; do not drop it.

**Step 4 Update every weight,  $w \leftarrow w - \eta \delta$  (its input)**

| Weight   | Working                       | Old    | New                |
|----------|-------------------------------|--------|--------------------|
| $w_{35}$ | $-1.2 - 0.1(0.12738)(0.5250)$ | $-1.2$ | $\mathbf{-1.2067}$ |
| $w_{45}$ | $1.1 - 0.1(0.12738)(0.8808)$  | $1.1$  | $\mathbf{1.0888}$  |
| $b_5$    | $-0.3 - 0.1(0.12738)$         | $-0.3$ | $\mathbf{-0.3127}$ |
| $w_{13}$ | $0.5 - 0.1(-0.03812)(1)$      | $0.5$  | $\mathbf{0.5038}$  |
| $w_{23}$ | $0.4 - 0.1(-0.03812)(1)$      | $0.4$  | $\mathbf{0.4038}$  |
| $b_3$    | $-0.8 - 0.1(-0.03812)$        | $-0.8$ | $\mathbf{-0.7962}$ |
| $w_{14}$ | $0.9 - 0.1(0.01471)(1)$       | $0.9$  | $\mathbf{0.8985}$  |
| $w_{24}$ | $1.0 - 0.1(0.01471)(1)$       | $1.0$  | $\mathbf{0.9985}$  |
| $b_4$    | $0.1 - 0.1(0.01471)$          | $0.1$  | $\mathbf{0.0985}$  |

**Answer.**  $y_5$  moved from 0.5097 towards the target 0, and every one of the nine parameters changed in the direction that reduces  $E$ . Repeat for the other three XOR patterns to complete one epoch.

**Three checks before you stop**

- **The output-layer weights move most.** They are nearest the error, and  $\delta$  shrinks as it is multiplied by weights and by  $h(1 - h) \leq 0.25$  on the way back. Ten layers of that is the **vanishing gradient** problem, named in §7.3.
- Every  $\Delta w$  here is under 0.02, which is what  $\eta = 0.1$  is supposed to give. A change of order 1 means a sign or a factor is wrong.
- Nothing outside the weights and biases changed. If your answer alters an activation or an input, you have misread the algorithm.

**7.N.8 Problem 2.3 – design a Hopfield network and recall from noise**

**Design a Hopfield network to store three binary patterns of length 4. Compute the weight matrix and demonstrate recall from a noisy input** 6

82 Bh 6 · the paper chooses no patterns, so choose three and say why · three patterns in four units is above the usual  $0.15N$  capacity, so pick orthogonal ones – that choice is the first mark

QUESTION AS PRINTED 82 Bh · Q7 · 6

Design a Hopfield network to store three binary patterns of length 4. Show how to compute the weight matrix and demonstrate the network's ability to recall a pattern from a noisy input.

**Step 1 Choose the patterns, and convert to bipolar**

|       | Binary  | Bipolar $s$    |
|-------|---------|----------------|
| $P_1$ | 1 1 1 1 | [1, 1, 1, 1]   |
| $P_2$ | 1 0 1 0 | [1, -1, 1, -1] |
| $P_3$ | 1 1 0 0 | [1, 1, -1, -1] |

- Convert with  $s_i = 2b_i - 1$ : a 1 stays +1, a 0 becomes -1. The training rule needs bipolar values or the outer products come out wrong.
- Why these three:** they are mutually **orthogonal**,  $P_i \cdot P_j = 0$  for  $i \neq j$ . Check:  $P_1 \cdot P_2 = 1 - 1 + 1 - 1 = 0$ ,  $P_1 \cdot P_3 = 1 + 1 - 1 - 1 = 0$ ,  $P_2 \cdot P_3 = 1 - 1 - 1 + 1 = 0$ .
- Orthogonal patterns do not interfere, so all three are stable even though  $3 > 0.15 \times 4$ . **Say this:** it is the design decision the question is testing.

**Step 2 The weight matrix**

$$w_{ij} = \sum_{p=1}^3 s_i^{(p)} s_j^{(p)} \quad (i \neq j), \quad w_{ii} = 0$$

- Worked entry,  $w_{12}$ :  
 $(1)(1) + (1)(-1) + (1)(1) = 1 - 1 + 1 = \mathbf{1}$ .
- Worked entry,  $w_{14}$ :  
 $(1)(1) + (1)(-1) + (1)(-1) = 1 - 1 - 1 = \mathbf{-1}$ .
- $w_{ji} = w_{ij}$ , so only six entries have to be computed.

$$W = \begin{bmatrix} 0 & 1 & 1 & -1 \\ 1 & 0 & -1 & 1 \\ 1 & -1 & 0 & 1 \\ -1 & 1 & 1 & 0 \end{bmatrix}$$

**Step 3 Check that all three patterns are stable**

$net_i = \sum_j w_{ij} s_j$ , then  $y_i = +1$  if  $net_i > 0$ ,  $-1$  if  $net_i < 0$ .

|       | net vector     | Sign             |
|-------|----------------|------------------|
| $P_1$ | [1, 1, 1, 1]   | [1, 1, 1, 1] ✓   |
| $P_2$ | [1, -1, 1, -1] | [1, -1, 1, -1] ✓ |
| $P_3$ | [1, 1, -1, -1] | [1, 1, -1, -1] ✓ |

Sample working for  $P_2$ , unit 1:

$net_1 = 0(1) + 1(-1) + 1(1) + (-1)(-1) = -1 + 1 + 1 = 1 > 0 \rightarrow +1$ , which is what  $P_2$  has. Every pattern reproduces itself, so all three are stable states.

**Step 4 Recall from a noisy input**

- Corrupt  $P_2 = 1010$  in its **first bit**: present  $x = 0010$ , i.e.  $y = [-1, -1, 1, -1]$ .

| Update | $net_i = \sum_j w_{ij} y_j$   | New $y_i$          | State          |
|--------|-------------------------------|--------------------|----------------|
| $y_1$  | $1(-1) + 1(1) + (-1)(-1) = 1$ | +1<br><b>flips</b> | [1, -1, 1, -1] |
| $y_2$  | $1(1) + (-1)(1) + 1(-1) = -1$ | -1                 | unchanged      |
| $y_3$  | $1(1) + (-1)(-1) + 1(-1) = 1$ | +1                 | unchanged      |
| $y_4$  | $(-1)(1) + 1(-1) + 1(1) = -1$ | -1                 | unchanged      |

**Answer.** The state converged to [1, -1, 1, -1], i.e. binary **1010** =  $P_2$ . The corrupted bit was repaired in one update, and a second sweep changes nothing, so the network has converged.

- Update one unit at a time and use the new value immediately** – that is what *asynchronous* means, and updating all four together is the commonest error on this question.
- Two honest remarks worth a mark each: this net also has **spurious stable states** it was never taught (the complement of every stored pattern is always one), and four units cannot reliably hold three patterns unless they are orthogonal, which is why they were chosen that way in step 1.

**7.N.9 Problem 3.1 – syntactic analysis and a parse tree****Explain syntactic analysis and draw a parse tree for the given sentence** 5+4

76 Bh 5+4 · the only NLP question in 41 papers that asks for something *drawn* · the sentence is four words and the tree has eight nodes, so this is four marks in three minutes

**QUESTION AS PRINTED** 76 Bh · Q9 · 5+4

Explain the steps of Natural Language Processing. Explain syntactic analysis and draw a parse tree for the following sentence. "A girl has a name."



**Step 1 Tag every word**

| Word | Part of speech       | Symbol  |
|------|----------------------|---------|
| A    | article (determiner) | Article |
| girl | noun                 | Noun    |
| has  | verb                 | Verb    |
| a    | article              | Article |
| name | noun                 | Noun    |

**Step 2 Write the grammar rules you will use**

- Sentence -> Noun\_phrase Verb\_phrase
- Noun\_phrase -> Article Noun
- Verb\_phrase -> Verb Noun\_phrase
- Article -> a | the      Noun -> girl | name  
Verb -> has
- **Print the rules beside the tree.** Each level of the tree is one rule application, and showing the rules is half of what “explain syntactic analysis” means.

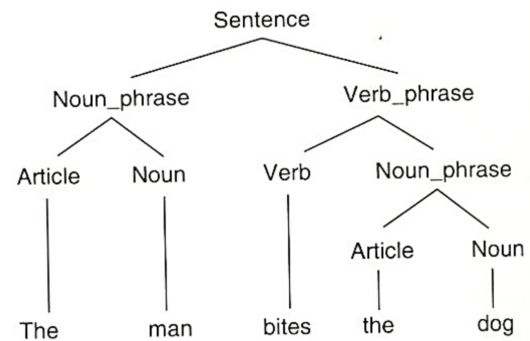
**Step 3 The parse tree**

```

Sentence
|-- Noun_phrase
|  |-- Article ---- "A"
|  `-- Noun ----- "girl"
`-- Verb_phrase
    |-- Verb ----- "has"
    `-- Noun_phrase
        |-- Article - "a"
        `-- Noun ---- "name"

```

Draw it as a tree with the sentence's words along the bottom. The published figure below is the same grammar on “*The man bites the dog*”, which is the same shape node for node – copy its layout and change the leaves.



**Figure 10.3** Parse tree for the sentence “The man bites the dog.”

- **Two checks:** every word of the sentence appears as exactly one leaf, in the original order; and every internal node is the left-hand side of one grammar rule whose right-hand side is its children.
- **Syntactic analysis in three lines**, which is the other half of the marks: it takes a tagged sentence and produces a representation of its **grammatical structure**; a **grammar** says which parts of speech exist and how they combine into phrases; the **parse tree** records which rule was applied where. It **rejects** “*The school goes to boy*”, and it **accepts** “*Colourless green ideas sleep furiously*”, which semantic analysis then rejects. Those two examples together define what syntax does and does not do.

**Sources and verification**

- Question text and the network figure are transcribed from AI/ocr/CT-653\_CT-710\_CT-78506\_NewScan\_OCR.md and AI/ocr/CT-653\_CT-
- Method follows BA Sir’s CH-07 worked examples (the Hebb net, the Adaline run and the back-propagation step) and *Insights on Artificial Intelligence* §7.1.10 for the Hopfield training and testing algorithm, checked independently.
- **Every number on these pages is recomputed by `verify.py`:** both perceptron runs epoch by epoch, the Hebb weights, an exhaustive search proving no integer weights realise XOR, the two-layer XOR truth table, the whole Hopfield weight matrix with all three stability checks and the recall sweep, the forward propagation, and all nine back-propagation updates.

## Abbreviations used in these notes

Only these are ever shortened. Anything not on this list is written out in full.

| Short | Means                                                | Short   | Means                                                                      |
|-------|------------------------------------------------------|---------|----------------------------------------------------------------------------|
| AI    | artificial intelligence                              | ANI     | artificial narrow intelligence                                             |
| AGI   | artificial general intelligence                      | ASI     | artificial super intelligence                                              |
| KR    | knowledge representation                             | KRR     | knowledge representation and reasoning                                     |
| PEAS  | performance measure, environment, actuators, sensors | GPS     | General Problem Solver                                                     |
| NLP   | natural language processing                          | NLU     | natural language understanding                                             |
| NLG   | natural language generation                          | CAPTCHA | completely automated public Turing test to tell computers and humans apart |
| ML    | machine learning                                     | ANN, NN | artificial neural network                                                  |
| MLP   | multilayer perceptron                                | BP      | back-propagation                                                           |
| SOM   | self-organizing map                                  | GA      | genetic algorithm                                                          |
| FIS   | fuzzy inference system                               | ID3     | Iterative Dichotomiser 3                                                   |
| CSP   | constraint satisfaction problem                      | DFS     | depth first search                                                         |
| BFS   | breadth first search                                 | DLS     | depth limited search                                                       |
| IDS   | iterative deepening search                           | UCS     | uniform cost search                                                        |
| CNF   | conjunctive normal form                              | WFF     | well-formed formula                                                        |
| FOL   | first-order logic                                    | FOPL    | first-order predicate logic                                                |
| RRS   | resolution refutation system                         | BBN     | Bayesian belief network                                                    |

| Short   | Means                      | Short  | Means                                                                                       |
|---------|----------------------------|--------|---------------------------------------------------------------------------------------------|
| ES      | expert system              | KB     | knowledge base                                                                              |
| CD      | conceptual dependency      | LISP   | list processing (the language)                                                              |
| MOP     | memory organization packet | isa    | the class-to-class link of a semantic net ( <i>instance</i> is the individual-to-class one) |
| attr(s) | attribute(s)               | repo   | repository                                                                                  |
| freq    | frequency                  | config | configuration                                                                               |
| info    | information                | no.    | number                                                                                      |
| Eg      | example                    | Vs     | compare / differentiate                                                                     |
| w/, w/o | with, without              | +Fig   | draw the figure                                                                             |
| →       | leads to, therefore        | =      | is, equals                                                                                  |

## Reading the year tags and chips

**Exam-paper month codes:** Ba = Baishakh · Jth = Jestha · Shr = Shrawan · Bh = Bhadra · Ash = Ashwin · Ka = Kartik · Po = Poush · Ma = Magh · Ch = Chaitra. Years are Bikram Sambat, so 80 Ch is 2080 Chaitra.

**Bold** year tags are Regular exams, plain are Back exams. A header printing “Regular / Back” in one box counts as Regular.

**Three course codes, three faces.** Plain = CT 653 (BCT, III-II, 26 papers) · monospace = CT 710 (BEI, IV-I, 7 papers) · *italic* = CT 78506 (BEX, IV-II, 8 papers). **The same year code names different papers under different codes:** 77 Ch and 77 Ch are two different exams, and only the font tells them apart.

**Tier chips** over all 41 papers, 2069–2082 BS: **TOP /41** = topic asked in **8 or more** papers · **HOT /41** = **4 to 7** papers · **PIN /41** = **1 to 3** papers but worth  $\geq 6$  marks.

**Mark chips** such as **2+6** are the marks that question actually carried. **syllabus, no PYQ yet** marks a syllabus topic never yet examined but written up anyway. **verified/added** marks anything researched and added that no source note in this folder covers.